



Secure One Million Users Readiness Report

Target: Ecommerce-style cloud architecture scenario

Prepared By: Anyaugo Space-Tech LTD

Assessment Type: Cloud Architecture, Security Automation, Load Testing, Monitoring, and Backup Readiness

Document Classification: Public Portfolio Sample / Capability Demonstration

Prepared For: Portfolio Review / Technical Assessment

Date: September 10, 2026

Assessment Environment Note

This portfolio demo was completed in a clean lab workstation environment. Tools were installed and version-checked during the assessment workflow so the evidence is reproducible, auditable, and easy to verify. No unauthorized production systems were tested.

Portfolio Boundary

This report demonstrates cloud-readiness thinking and controlled local tool execution. It is not a live one-million-user production stress test and does not represent a scan of any real company environment. Live validation would require written authorization, staging infrastructure, monitoring, traffic limits, and a stop plan.

Executive Summary

Project 2 demonstrates the combined niche of cybersecurity plus cloud architecture. The evidence shows how an ecommerce platform could be designed, reviewed, scanned, hardened, load-tested, monitored, and protected with backups using a controlled lab workflow.

Area	Evidence	Result
Architecture	Traffic flow and one-million-user readiness design	Clear design for DNS, CDN/WAF, load balancing, autoscaling, cache, database, queues, monitoring, and backups.
Infrastructure as Code	Terraform design, version proof, init, and validation	Terraform configuration validated successfully in a Linux lab environment without applying real cloud changes.
Cloud Security Review	Checkov scan against demo Terraform	Real scan found intentional risks and mapped them to remediation priorities.
Server Hardening	Ansible playbook, version proof, and local check-mode run	Hardening workflow documented and verified in a controlled lab.
Container and Dependencies	Trivy scan	Real scan detected dependency vulnerabilities and Dockerfile hardening issues in a demo target.
Secrets Detection	Gitleaks scan	Real scan detected a fake demonstration secret and showed redaction workflow.
Load Testing	k6 local load test	Controlled local test completed with successful checks and zero failed HTTP requests in the captured run.
Operations	Health check plus backup and restore verification	Local health-check and backup-restore workflow were proven with screenshots.

Key Positioning Statement

The project does not claim unauthorized access to any production system. It proves a repeatable assessment method: design the target architecture, validate infrastructure code, scan for risk, harden servers, detect secrets, test controlled load, and verify operations readiness.

Architecture and Readiness Design

The architecture uses layered defense and scaling: users connect through DNS, CDN/WAF, load balancing, application servers or autoscaling groups, cache, database cluster, background queues, workers, object storage, logs, metrics, alerts, and backups.

Layer	Purpose	Security or resilience value
DNS and CDN/WAF	Route traffic and absorb/filter edge traffic	DDoS protection, caching, bot controls, WAF rules
Load Balancer	Distribute requests across healthy app instances	TLS policy, health checks, access logging
App Servers / ASG	Serve ecommerce workloads across scalable capacity	Hardened images, patching, least privilege, secret handling
Cache and Database	Reduce repeated reads and persist critical data	Encryption, private networking, backups, audit logs
Queue and Workers	Move slow jobs away from the live request path	Retry control, dead-letter handling, IAM restrictions
Monitoring and Backups	Detect incidents and restore data	Alerts, runbooks, retention, restore verification

Tool Evidence Summary

Test	Tool / Method	What was proven	Honest boundary
1	Architecture design	One-million-user readiness model and traffic flow	Design evidence, not live production load
2	Terraform	Infrastructure code structure, tool install, init, and validation	No terraform apply or real cloud deployment
3	Checkov	Terraform security scan and remediation priorities	Demo Terraform baseline used intentionally
4	Ansible	Hardening plan, tool install, and local check mode	No unauthorized server changes
5	Trivy	Dependency and Dockerfile scan result	Controlled local vulnerable demo target
6	Gitleaks	Secret detection workflow with redaction	Fake demonstration secret only
7	k6	Controlled load test against local API	Not a live million-user stress test
8	Python and shell checks	Health-check and backup-restore verification	Local operational proof, not production monitoring access

How this maps to client work

In an authorized company environment, this workflow would be adapted to the client's approved scope: review existing architecture, scan infrastructure-as-code, validate server hardening, scan containers and repositories, run approved load tests in staging, connect monitoring and alerting, and verify backup/restore readiness.

Thirty-Day Client Roadmap

Period	Workstream	Deliverable
Days 1-5	Discovery and authorization	Scope, access boundaries, architecture inventory, risk assumptions, test plan
Days 6-10	Infrastructure and security review	Terraform/IaC review, Checkov findings, remediation priority map
Days 11-15	Hardening and dependency review	Ansible hardening plan, Trivy scan, Gitleaks scan, quick wins
Days 16-20	Performance and scale validation	Approved k6/Locust test plan, bottleneck analysis, safe capacity assumptions
Days 21-25	Monitoring and backup readiness	Dashboard/alert plan, backup policy, restore verification, runbooks
Days 26-30	Executive report and retest plan	Final report, risk register, monthly support proposal, retest scope

Monthly continuation can include vulnerability scanning, IaC policy checks, dependency scanning, secrets monitoring, patch/hardening review, load-test planning, uptime/alert review, backup restore drills, and executive risk reporting.

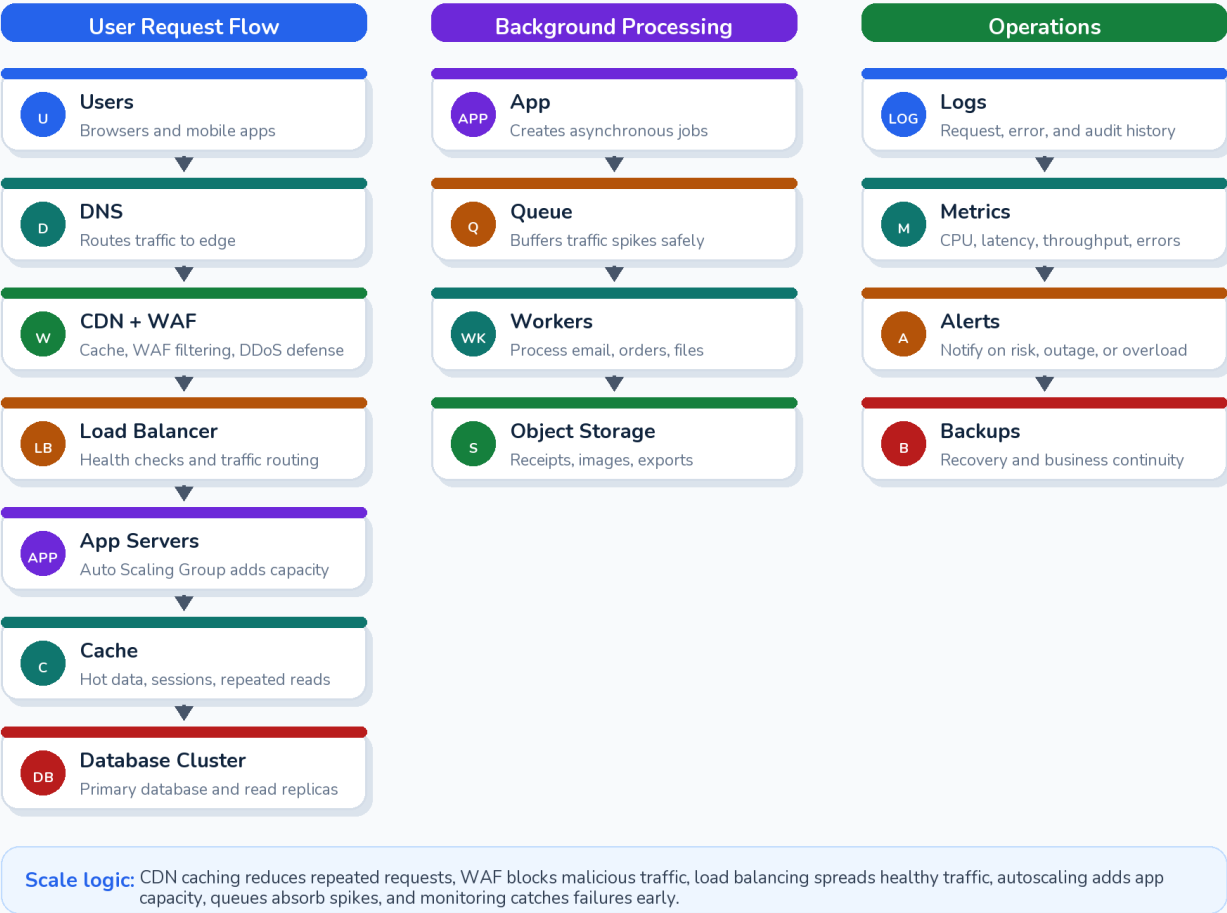
Evidence Appendix

The following pages include the screenshot evidence captured during the controlled Project 2 lab workflow.

Test 1A: One Million Users Architecture Diagram

Secure One Million Users Readiness Demo

Ecommerce-style cloud architecture proving scale, security controls, automation, and operations visibility.



C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-1-Architecture-Diagram-and-Traffic-Flow\Finding1A-One-Million-Users-Architecture-Diagram.png

Test 1B: Traffic Flow Explanation

Traffic Flow Explanation

How the secure one million users architecture handles traffic, bottlenecks, and security risks.

Main idea

One million users should not hit one backend directly.

Scale method

Cache, distribute, autoscale, queue, and observe.

Security method

Filter early, isolate systems, log activity, and recover.

Layer-by-Layer Flow

Layer	What happens	How it scales	Security or failure control
Users	Customers connect from browsers and mobile apps.	Traffic arrives globally and unevenly, so the platform must handle bursts.	TLS, bot controls, rate limits, and safe authentication protect entry points.
DNS	Routes users to the nearest healthy edge or provider endpoint.	Supports failover and reduces dependency on one origin location.	DNSSEC, short TTL strategy, and provider resilience reduce routing risk.
CDN + WAF	Caches static content and filters harmful requests before origin.	Stops repeated static requests from hitting the app servers directly.	WAF rules, DDoS protection, bot filtering, and geo/IP controls reduce attack load.
Load Balancer	Distributes valid traffic across healthy application servers.	Prevents one server from becoming the single pressure point.	Health checks, TLS policy, access logs, and security groups protect routing.
App Servers / ASG	Runs the ecommerce backend across multiple instances or containers.	Auto Scaling adds or removes capacity based on demand.	Hardened images, patching, least privilege, and secrets management reduce compromise risk.
Cache	Stores hot reads such as sessions, product lists, and repeated queries.	Protects the database from avoidable read pressure.	Private networking, encryption, auth, and expiry policies protect cached data.
Database Cluster	Stores orders, users, inventory, payments metadata, and application state.	Read replicas, indexing, connection pooling, and backups improve resilience.	Encryption, backups, access control, audit logs, and patching protect critical data.
Queue + Workers	Moves slow jobs away from the live user request path.	Absorbs spikes from email, order processing, exports, and file tasks.	Job validation, retries, dead-letter queues, and IAM controls prevent unsafe processing.
Observability	Collects logs, metrics, alerts, traces, and backup status.	Shows bottlenecks before users experience major failure.	Alerting, audit trails, incident response, and recovery tests support operations.

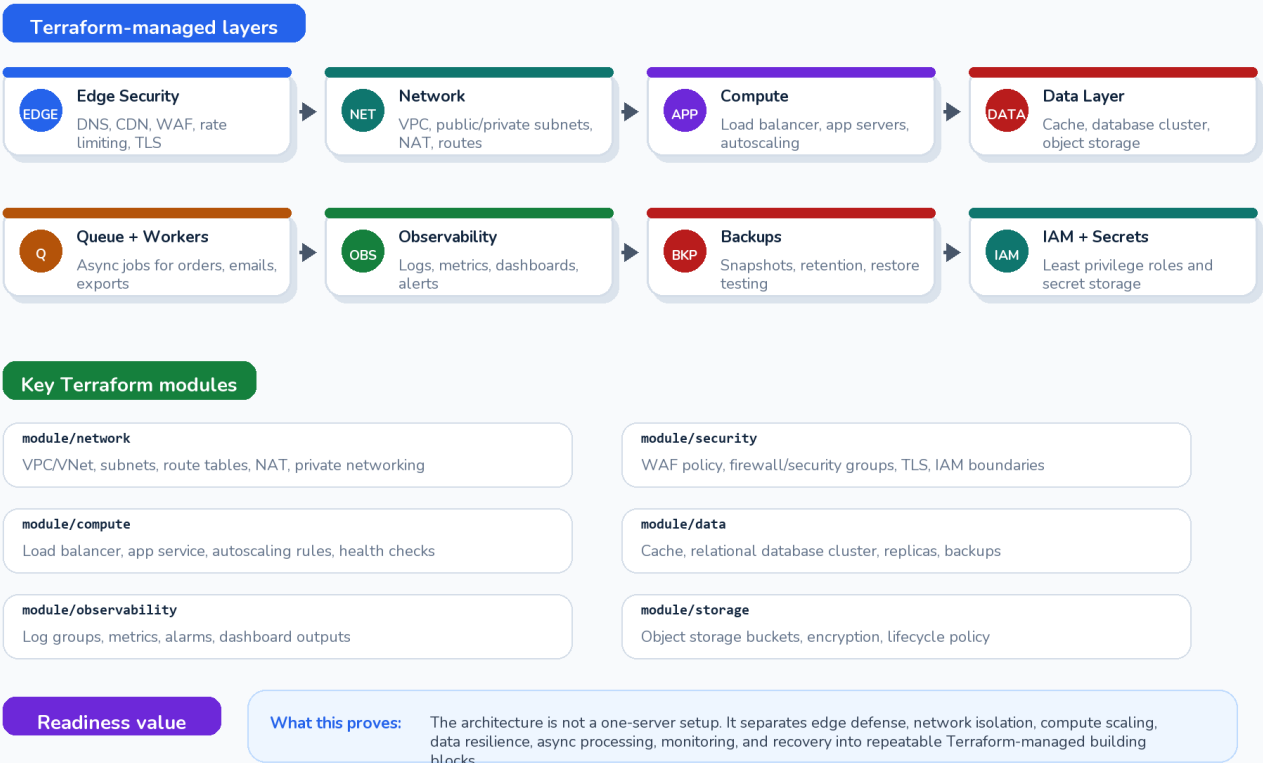
Portfolio statement: This design demonstrates readiness thinking. Real one million-user capacity must be validated with client traffic patterns, load testing, production telemetry, and cloud cost limits.

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-1-Architecture-Diagram-and-Traffic-Flow\Finding1B-Traffic-Flow-Explanation.png

Test 2A: Terraform Infrastructure Blueprint

Terraform Infrastructure Design Blueprint

Repeatable cloud infrastructure design for a secure ecommerce platform prepared for heavy traffic.



C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-2-Terraform-Infrastructure-Design\Finding2A-Terraform-Infrastructure-Blueprint.png

Test 2B: Terraform Design Structure and Workflow

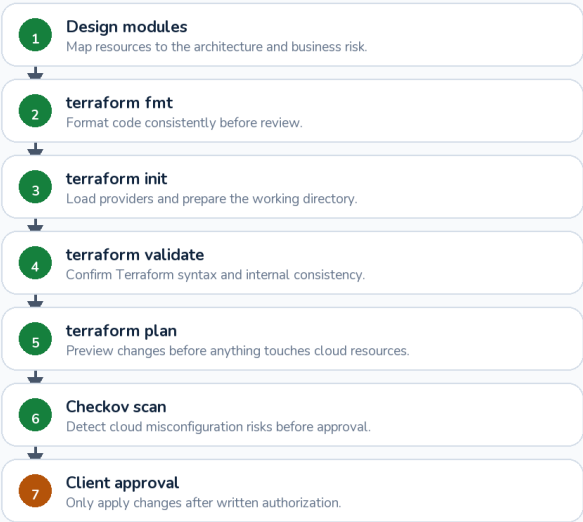
Terraform Design Structure and Safe Workflow

How the infrastructure design should be organized, reviewed, scanned, and approved before deployment.

Recommended folder layout

```
terraform/  
  
environments/  
  prod/  
    main.tf  
    providers.tf  
    variables.tf  
    outputs.tf  
    terraform.tfvars.example  
  
modules/  
  network/main.tf  
  security/main.tf  
  compute/main.tf  
  data/main.tf  
  observability/main.tf  
  storage/main.tf
```

Safe Terraform workflow



One million-user settings

App capacity	Min 6 instances, max 60, CPU/latency scaling policy	Availability	Multi-AZ public edge and private application/data tiers
Database	Primary database, read replicas, automated backups, encryption	Performance	CDN caching, Redis cache, queue workers, connection pooling
Security	WAF rules, private subnets, least privilege, secret storage	Operations	Logs, metrics, alerts, backup checks, incident response hooks

Portfolio note: This is an infrastructure design proof. Actual deployment requires a client cloud account, budget limits, and written authorization.

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-2-Terraform-Infrastructure-Design\Finding2B-Terraform-Design-Structure-and-Workflow.png

Test 2C: Terraform Version Installed

```
user@DESKTOP-PCAD3H9: /n × + ~
2026-09-09 21:49:22 (70.7 MB/s) - written to stdout [3980/3980]

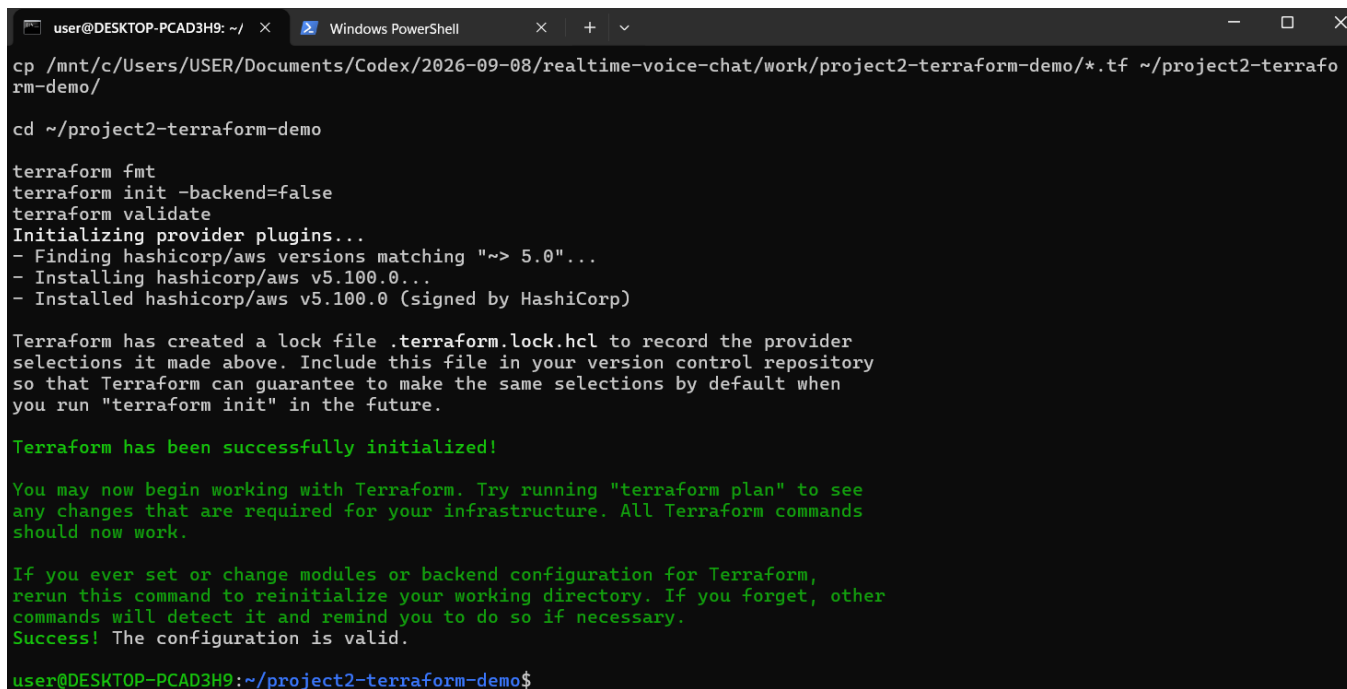
deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com resolute main
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:4 https://apt.releases.hashicorp.com resolute InRelease [12.9 kB]
Hit:5 http://security.ubuntu.com/ubuntu resolute-security InRelease
Get:6 https://apt.releases.hashicorp.com resolute/main amd64 Packages [316 kB]
Fetched 329 kB in 11s (28.7 kB/s)
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  terraform

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2
  Download size: 35.8 MB
  Space needed: 120 MB / 1023 GB available

Get:1 https://apt.releases.hashicorp.com resolute/main amd64 terraform amd64 1.16.2-1 [35.8 MB]
Fetched 35.8 MB in 18s (2017 kB/s)
Selecting previously unselected package terraform.
(Reading database ... 47281 files and directories currently installed.)
Preparing to unpack .../terraform_1.16.2-1_amd64.deb ...
Unpacking terraform (1.16.2-1) ...
Setting up terraform (1.16.2-1) ...
Terraform v1.16.2
on linux_amd64
user@DESKTOP-PCAD3H9: /mnt/c/Users/USER/Documents/Codex/2026-09-08/realtime-voice-chat/work/project2-ansible-hardening$
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-2-Terraform-Infrastructure-Design\Finding2C-Terraform-Version-Installed.png

Test 2D: Terraform Init and Validate



```
user@DESKTOP-PCAD3H9: ~/ x Windows PowerShell x + v
cp /mnt/c/Users/USER/Documents/Codex/2026-09-08/realtime-voice-chat/work/project2-terraform-demo/*.tf ~/project2-terrafo
rm-demo/

cd ~/project2-terraform-demo

terraform fmt
terraform init -backend=false
terraform validate
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
Success! The configuration is valid.

user@DESKTOP-PCAD3H9:~/project2-terraform-demo$
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-2-Terraform-Infrastructure-Design\Finding2D-Terraform-Init-and-Validate.png

Test 3A: Checkov Terraform Scan Result

Checkov Terraform Security Scan Result

Actual IaC scan against the Project 2 Terraform demo baseline before deployment.

12

Passed checks

32

Failed checks

0

Skipped checks

Scan command

```
> checkov -d project2-terraform-demo --framework terraform --compact

By Prisma Cloud | version: 3.3.16
terraform scan results: Passed checks: 12, Failed checks: 32, Skipped checks: 0
```

Notable failed checks

Check ID	Risk detected	Resource	Lines
CKV_AWS_53	Ensure S3 bucket has block public ACLS enabled	aws_s3_bucket_public_access_block.site_assets	20-26
CKV_AWS_54	Ensure S3 bucket has block public policy enabled	aws_s3_bucket_public_access_block.site_assets	20-26
CKV_AWS_24	Ensure no security groups allow ingress from 0.0.0.0 to port 22	aws_security_group.app	28-55
CKV_AWS_17	Ensure all data stored in RDS is not publicly accessible	aws_db_instance.orders	57-70
CKV_AWS_16	Ensure all data stored in the RDS is securely encrypted at rest	aws_db_instance.orders	57-70
CKV_AWS_133	Ensure that RDS instances has backup policy	aws_db_instance.orders	57-70
CKV_AWS_157	Ensure that RDS instances have Multi-AZ enabled	aws_db_instance.orders	57-70
CKV_AWS_29	Ensure all data stored in the ElastiCache Replication Group is securely encrypted at rest	aws_elasticache_replication_group.session_cache	72-81
CKV_AWS_30	Ensure all data stored in the ElastiCache Replication Group is securely encrypted at transit	aws_elasticache_replication_group.session_cache	72-81
CKV2_AWS_50	Ensure AWS ElastiCache Redis cluster with Multi-AZ Automatic Failover feature set to enabled	aws_elasticache_replication_group.session_cache	72-81

Interpretation: The baseline intentionally contains public access, weak network exposure, missing encryption, missing backups, and missing high-availability controls. These must be remediated before client approval or deployment.

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-3-Checkov-Terraform-Security-Scan\Finding3A-Checkov-Terraform-Scan-Result.png

Test 3B: Checkov Remediation Priority Map

Checkov Remediation Priority Map

How the failed Terraform checks translate into secure cloud architecture improvements.

Remediation priorities

Area	Risk found	Fix before deployment	Business value
S3 / Object Storage	Public access block disabled, versioning/logging/lifecycle controls missing.	Enable public access block, KMS encryption, versioning, access logs, lifecycle policy, and replication where required.	Protects customer files, reports, receipts, exports, and static assets.
Security Group	SSH from 0.0.0.0/0 and unrestricted egress detected.	Remove public SSH, use VPN/bastion/SSM, restrict egress, document every rule, and attach groups only where required.	Reduces direct server exposure and lateral movement risk.
Database	RDS is public, unencrypted, no backups, no Multi-AZ, weak logging/monitoring posture.	Private subnet only, encryption at rest, backup retention, Multi-AZ, deletion protection, logs, performance insights, and IAM auth where suitable.	Protects order, user, inventory, and transaction records.
Cache	Redis/ElastiCache lacks transit encryption, at-rest encryption, auth token, and automatic failover.	Enable TLS in transit, encryption at rest, auth token/ACLs, Multi-AZ failover, and private network placement.	Protects sessions, hot product data, and application performance under load.
Approval Workflow	Terraform must not be applied straight from a local laptop into production.	Require pull request review, Checkov scan, Terraform plan review, change approval, and rollback plan before apply.	Improves governance and prevents unmanaged production changes.

Report wording to use

Checkov was used to scan the Terraform demo baseline before deployment. The scan identified infrastructure risks that must be remediated before applying changes to any client cloud environment.

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-3-Checkov-Terraform-Security-Scan\Finding3B-Checkov-Remediation-Priority-Map.png

Test 4A: Ansible Playbook Structure

Ansible Server Hardening Playbook Structure

Repeatable server configuration blueprint for ecommerce application servers after infrastructure provisioning.

Ansible project layout

project2-ansible-hardening/

```
inventory/
  prod.ini
group_vars/
  web.yml
playbooks/
  harden-web-servers.yml
roles/
  common_hardening/
    tasks/main.yml
    handlers/main.yml
README.md
```

Safe execution workflow

- 1 Confirm written authorization and exact host scope.
- 2 Review inventory and server groups before any run.
- 3 Run syntax check and dry-run mode with --check --diff.
- 4 Review proposed changes with the client or senior engineer.
- 5 Apply during an approved maintenance/change window.
- 6 Verify SSH, firewall, patching, services, logs, and alerts.

Example commands

```
> ansible-playbook -i inventory/prod.ini playbooks/harden-web-servers.yml --syntax-check
> ansible-playbook -i inventory/prod.ini playbooks/harden-web-servers.yml --check --diff
> ansible-playbook -i inventory/prod.ini playbooks/harden-web-servers.yml
Note: apply only after written authorization and approved change window.
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-4-Ansible-Server-Hardening-Plan\Finding4A-Ansible-Playbook-Structure.png

Test 4B: Ansible Server Hardening Control Matrix

Ansible Server Hardening Control Matrix

Security controls that would be applied consistently across authorized Linux application servers.

Hardening controls

Control	What Ansible changes	Security value	Ansible evidence
Patch management	Install security baseline packages and enable unattended security updates.	Reduces exposure to known vulnerabilities.	apt, unattended-upgrades
SSH hardening	Disable direct root login and password-based SSH authentication.	Reduces brute-force and credential-stuffing risk.	lineinfile, handler
Firewall baseline	Allow only approved SSH source range and HTTPS application traffic.	Limits exposed network surface on app servers.	community.general.ufw
Service assurance	Ensure auditd and fail2ban are installed, enabled, and running.	Improves host visibility and blocks repeated abuse.	service
Configuration safety	Validate SSH configuration before restarting the service.	Prevents lockout caused by invalid SSH settings.	validate, notify
Repeatability	Use inventory, group variables, roles, handlers, check mode, and diff mode.	Makes hardening reviewable and repeatable across many servers.	roles, --check, --diff

Portfolio wording: Ansible was used here as a hardening design blueprint. Actual execution against company servers requires written authorization, approved credentials, inventory confirmation, dry-run review, and a rollback plan.

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-4-Ansible-Server-Hardening-Plan\Finding4B-Ansible-Server-Hardening-Control-Matrix.png

Test 4C: Ansible Version Installed

```
user@DESKTOP-PCAD3H9: /mnt/c/Users/USER$ source ~/.bashrc
pipx install --include-deps ansible
ansible --version
installed package ansible 14.4.0, installed using Python 3.14.4
These apps are now globally available
- ansible
- ansible-community
- ansible-config
- ansible-console
- ansible-doc
- ansible-galaxy
- ansible-inventory
- ansible-playbook
- ansible-pull
- ansible-test
- ansible-vault
- cffi-gen-src
done!
ansible [core 2.21.4]
  config file = None
  configured module search path = ['/home/user/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /home/user/.local/share/pipx/venvs/ansible/lib/python3.14/site-packages/ansible
  ansible collection location = /home/user/.ansible/collections:/usr/share/ansible/collections
  executable location = /home/user/.local/bin/ansible
  python version = 3.14.4 (main, Aug 20 2026, 10:41:58) [GCC 15.2.0] (/home/user/.local/share/pipx/venvs/ansible/bin/python)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
user@DESKTOP-PCAD3H9: /mnt/c/Users/USER$
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-4-Ansible-Server-Hardening-Plan\Finding4C-Ansible-Version-Installed.png

Test 4D: Ansible Local Check Mode Run

```
user@DESKTOP-PCAD3H9: /n  ×  +  v
12      msg: "Detected {{ ansible_distribution }} {{ ansible_distribution_version }} on {{ ansible_architecture }}."
      ^ column 14

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.

ok: [localhost] => {
  "msg": "Detected Ubuntu 26.04 on x86_64."
}

TASK [Check whether SSH configuration file exists] *****
ok: [localhost]

TASK [Report SSH configuration file status] *****
ok: [localhost] => {
  "msg": "SSH configuration present: False"
}

TASK [Check whether UFW is installed] *****
skipping: [localhost]

TASK [Report UFW availability] *****
ok: [localhost] => {
  "msg": "UFW command available: True"
}

PLAY RECAP *****
localhost      : ok=6    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

user@DESKTOP-PCAD3H9:/mnt/c/Users/USER/Documents/Codex/2026-09-08/realtime-voice-chat/work/project2-ansible-hardening$ |
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-4-Ansible-Server-Hardening-Plan\Finding4D-Ansible-Local-Check-Mode-Run.png

Test 5A: Trivy Version Installed

```
user@DESKTOP-PCAD3H9: ~/project2-terraform-demo$ apt-get install trivy
deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb generic main
Get:1 https://aquasecurity.github.io/trivy-repo/deb generic InRelease [3237 B]
Get:2 https://aquasecurity.github.io/trivy-repo/deb generic/main amd64 Packages [373 B]
Hit:3 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:6 https://apt.releases.hashicorp.com resolute InRelease [12.9 kB]
Get:7 https://apt.releases.hashicorp.com resolute/main amd64 Packages [317 kB]
Hit:8 http://security.ubuntu.com/ubuntu resolute-security InRelease
Fetched 333 kB in 12s (27.8 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following NEW packages will be installed:
  trivy
0 upgraded, 1 newly installed, 0 to remove and 2 not upgraded.
Need to get 50.4 MB of archives.
After this operation, 168 MB of additional disk space will be used.
Get:1 https://aquasecurity.github.io/trivy-repo/deb generic/main amd64 trivy amd64 0.74.0 [50.4 MB]
Fetched 50.4 MB in 12s (4379 kB/s)
Selecting previously unselected package trivy.
(Reading database ... 47284 files and directories currently installed.)
Preparing to unpack .../trivy_0.74.0_amd64v1.deb ...
Unpacking trivy (0.74.0) ...
Setting up trivy (0.74.0) ...
Version: 0.74.0
user@DESKTOP-PCAD3H9:~/project2-terraform-demo$
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-5-Trivy-Container-and-Dependency-Scan\Finding5A-Trivy-Version-Installed.png

Test 5B: Trivy Container and Dependency Scan Result

```
user@DESKTOP-PCAD3H9: ~/project2-trivy-demo$ trivy dockerfile Dockerfile (dockerfile)
Tests: 27 (SUCCESSES: 25, FAILURES: 2)
Failures: 2 (UNKNOWN: 0, LOW: 1, MEDIUM: 0, HIGH: 1, CRITICAL: 0)

DS-0002 (HIGH): Last USER command in Dockerfile should not be 'root'

Running containers with 'root' user can lead to a container escape situation. It is a best practice to run containers as non-root users, which can be done by adding a 'USER' statement to the Dockerfile.
See https://avd.aquasec.com/misconfig/ds-0002

Dockerfile:3
3 [ USER root

DS-0026 (LOW): Add HEALTHCHECK instruction in your Dockerfile

You should add HEALTHCHECK instruction in your docker container images to perform the health check on running containers.
See https://avd.aquasec.com/misconfig/ds-0026

user@DESKTOP-PCAD3H9:~/project2-trivy-demo$
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-5-Trivy-Container-and-Dependency-Scan\Finding5B-Trivy-Container-and-Dependency-Scan-Result.png

Test 6A: Gitleaks Version Installed

```
user@DESKTOP-PCAD3H9: ~/project2-gitleaks-demo$ gitleaks detect --source . --no-git --redact --verbose
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [143 kB]
Get:7 https://apt.releases.hashicorp.com resolute InRelease [12.9 kB]
Err:7 https://apt.releases.hashicorp.com resolute InRelease
  The following signatures couldn't be verified because the public key is not available: NO_PUBKEY FC9CA96ACA026560
Hit:8 http://security.ubuntu.com/ubuntu resolute-security InRelease
Fetched 894 kB in 11s (80.6 kB/s)
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
Warning: An error occurred during the signature verification. The repository is not updated and the previous index files
will be used. OpenPGP signature verification failed: https://apt.releases.hashicorp.com resolute InRelease: The followi
ng signatures couldn't be verified because the public key is not available: NO_PUBKEY FC9CA96ACA026560
Warning: Failed to fetch https://apt.releases.hashicorp.com/dists/resolute/InRelease The following signatures couldn't
be verified because the public key is not available: NO_PUBKEY FC9CA96ACA026560
Warning: Some index files failed to download. They have been ignored, or old ones used instead.
Installing:
  gitleaks

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2
  Download size: 2355 kB
  Space needed: 6790 kB / 1021 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 gitleaks amd64 8.16.0-1build2 [2355 kB]
Fetched 2355 kB in 3s (832 kB/s)
Selecting previously unselected package gitleaks.
(Reading database ... 47294 files and directories currently installed.)
Preparing to unpack .../gitleaks_8.16.0-1build2_amd64.deb ...
Unpacking gitleaks (8.16.0-1build2) ...
Setting up gitleaks (8.16.0-1build2) ...
version is set by build process
user@DESKTOP-PCAD3H9:~/project2-gitleaks-demo$ gitleaks detect --source . --no-git --redact --verbose
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-6-Gitleaks-Secrets-Detection\Finding6A-Gitleaks-Version-Installed.png

Test 6B: Gitleaks Secrets Detection Result

```

user@DESKTOP-PCAD3H9:~/project2-gitLeaks-demo$ gitleaks detect --source . --no-git --redact --verbose

      ○
     / \
    /   \
   ○     ○
  /       \
 ○         ○
/           \
gitleaks

Finding:      apiToken: "REDACTED",
Secret:       REDACTED
RuleID:       github-pat
Entropy:      0.668996
File:         app-config.js
Line:         4
Fingerprint:  app-config.js:github-pat:4

12:50AM INF scan completed in 11.5ms
12:50AM WRN leaks found: 1
user@DESKTOP-PCAD3H9:~/project2-gitLeaks-demo$ |
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-6-Gitleaks-Secrets-Detection\Finding6B-Gitleaks-Secrets-Detection-Result.png

Test 7A: k6 Version Installed

```
user@DESKTOP-PCAD3H9: ~/project2-gitleaks-demo$ |
Windows PowerShell
Err:5 https://apt.releases.hashicorp.com resolute InRelease
The following signatures couldn't be verified because the public key is not available: NO_PUBKEY FC9CA96ACA026560
Hit:7 http://security.ubuntu.com/ubuntu resolute-security InRelease
Get:8 https://dl.k6.io/deb stable/main amd64 Packages [7199 B]
Fetched 24.2 kB in 11s (2108 B/s)
Reading package lists... Done
W: An error occurred during the signature verification. The repository is not updated and the previous index files will
be used. OpenPGP signature verification failed: https://apt.releases.hashicorp.com resolute InRelease: The following sig
natures couldn't be verified because the public key is not available: NO_PUBKEY FC9CA96ACA026560
W: Failed to fetch https://apt.releases.hashicorp.com/dists/resolute/InRelease The following signatures couldn't be ver
ified because the public key is not available: NO_PUBKEY FC9CA96ACA026560
W: Some index files failed to download. They have been ignored, or old ones used instead.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following NEW packages will be installed:
  k6
0 upgraded, 1 newly installed, 0 to remove and 2 not upgraded.
Need to get 28.9 MB of archives.
After this operation, 65.2 MB of additional disk space will be used.
Get:1 https://dl.k6.io/deb stable/main amd64 k6 amd64 2.2.0 [28.9 MB]
Fetched 28.9 MB in 16s (1791 kB/s)
Selecting previously unselected package k6.
(Reading database ... 47301 files and directories currently installed.)
Preparing to unpack .../archives/k6_2.2.0_amd64.deb ...
Unpacking k6 (2.2.0) ...
Setting up k6 (2.2.0) ...
k6 v2.2.0 (commit/00a9a1b7f5, go1.26.5, linux/amd64)
user@DESKTOP-PCAD3H9:~/project2-gitleaks-demo$ |
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-7-Load-Testing-with-k6-or-Locust\Finding7A-k6-Version-Installed.png

Test 7B: Local Demo App Running

```
user@DESKTOP-PCAD3H9:~/project2-gitleaks-demo$ mkdir -p ~/project2-k6-demo
cp -r /mnt/c/Users/USER/Documents/Codex/2026-09-08/realtime-voice-chat/work/project2-k6-demo/* ~/project2-k6-demo/
cd ~/project2-k6-demo

python3 app.py
Project 2 k6 demo API running at http://127.0.0.1:8080
Endpoints: /health, /api/products, /api/cart
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-7-Load-Testing-with-k6-or-Locust\Finding7B-Local-Demo-App-Running.png

Test 7C: k6 Load Test Result

```
user@DESKTOP-PCAD3H9: ~/l × Command Prompt × + v

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 2166    53.622511/s
checks_succeeded...: 100.00% 2166 out of 2166
checks_failed.....: 0.00%  0 out of 2166

✓ health status is 200
✓ health response has ok
✓ products status is 200
✓ products returned
✓ cart status is 200
✓ cart has total

HTTP
http_req_duration.....: avg=16.57ms min=13.42ms med=16.47ms max=29.47ms p(90)=17.04ms p(95)=17.38ms
{ expected_response:true }...: avg=16.57ms min=13.42ms med=16.47ms max=29.47ms p(90)=17.04ms p(95)=17.38ms
http_req_failed.....: 0.00%  0 out of 1083
http_reqs.....: 1083    26.811256/s

EXECUTION
iteration_duration.....: avg=1.05s   min=1.04s   med=1.05s   max=1.06s   p(90)=1.05s   p(95)=1.05s
iterations.....: 361    8.937085/s
vus.....: 2    min=1    max=20
vus_max.....: 20    min=20    max=20

NETWORK
data_received.....: 274 kB  6.8 kB/s
data_sent.....: 85 kB  2.1 kB/s

running (0m40.4s), 00/20 VUs, 361 complete and 0 interrupted iterations
default ✓ [=====] 00/20 VUs  40s

C:\Users\USER>
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-7-Load-Testing-with-k6-or-Locust\Finding7C-k6-Load-Test-Result.png

Test 8A: Monitoring Alerts and Backup Plan

Monitoring, Alerts, and Backup Plan

Project 2 Evidence - Test 8 | Operations readiness for a scalable ecommerce platform

Monitoring • Health endpoint status • HTTP 5xx error rate • p95 response latency • CPU, memory, and disk • Database connections • Queue depth and cache hit rate	Alerts • API health fails for 2 minutes • 5xx error rate rises above 1% • p95 latency exceeds 500 ms • Database CPU remains above 80% • Backup or restore check fails • Abnormal WAF or auth activity	Backups • Encrypted object storage • Daily, weekly, monthly retention • Checksum recording • Scheduled restore tests • Least-privilege backup access • Documented recovery goals
---	--	---

Production Tool Mapping

Grafana or CloudWatch dashboards show service health and capacity. Prometheus or cloud-native metrics feed alert rules. Wazuh or a SIEM provides host and security-event visibility. Object storage with versioning, encryption, and lifecycle policies protects backups. Runbooks define triage, escalation, rollback, and recovery.

Portfolio Evidence Boundary

This portfolio uses a controlled local operations demo to prove health-check and backup-restore workflow. A client production implementation would require approved monitoring access, cloud accounts, alert destinations, backup storage policies, and written authorization.

Evidence sequence: 8A plan image | 8B local API running | 8C health-check result | 8D backup and restore verification

Anyango Space-Tech LTD - Public Portfolio Sample

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-8-Monitoring-Alerts-and-Backup-Plan\Finding8A-Monitoring-Alerts-and-Backup-Plan.png

Test 8B: Monitoring Health Endpoint Running

```
user@DESKTOP-PCAD3H9:~/project2-k6-demo$ cd ~/project2-k6-demo
python3 app.py
Project 2 k6 demo API running at http://127.0.0.1:8080
Endpoints: /health, /api/products, /api/cart
|
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-8-Monitoring-Alerts-and-Backup-Plan\Finding8B-Monitoring-Health-Endpoint-Running.png

Test 8C: Monitoring Health Check Result

```
C:\Users\USER>wsl -d Ubuntu -- bash -lc "mkdir -p ~/project2-ops-demo && cp -r /mnt/c/Users/USER/Documents/Codex/2026-09-08/realtime-voice-chat/work/project2-ops-demo/* ~/project2-ops-demo/ && cd ~/project2-ops-demo && python3 health_check.py"
Monitoring Health Check Result
target=http://127.0.0.1:8080/health
http_status=200
response_time_ms=62.79
service_status=ok
alert_state=OK
decision=no alert; service is reachable and within threshold

C:\Users\USER> |
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-8-Monitoring-Alerts-and-Backup-Plan\Finding8C-Monitoring-Health-Check-Result.png

Test 8D: Backup and Restore Verification

```
C:\Users\USER>wsl -d Ubuntu -- bash -lc "cd ~/project2-ops-demo && bash backup_restore_check.sh"
Backup and Restore Verification Result
backup_file=/home/user/project2-ops-demo/backups/orders-backup-20260910-033338.tar.gz
checksum_file=/home/user/project2-ops-demo/backups/orders-backup-20260910-033338.tar.gz.sha256
source_hash=b96809f6a2753af9d0be5f52f885f07f89eceb33cb20ec0cc40b6d043c008ab3
restore_hash=b96809f6a2753af9d0be5f52f885f07f89eceb33cb20ec0cc40b6d043c008ab3
restore_state=SUCCESS
decision=backup artifact restored correctly and data integrity matched
C:\Users\USER>
```

C:\Users\USER\OneDrive\Desktop\Juice_Shop_Assessment_Portfolio\Project
2\Evidence-Screenshots\Test-8-Monitoring-Alerts-and-Backup-Plan\Finding8D-Backup-and-Restore-Verification.png